

Proyecto Final: Consola de videojuegos Retro

24 de mayo de 2026

Repositorio del código fuente (GitHub): <https://github.com/daniel13nic/proyecto-retro-embebidos.git>

Video demostrativo (YouTube): https://youtu.be/Xch_X6m8AQw

1. Objetivo

El objetivo de este proyecto es diseñar e implementar un sistema embebido que opere como una consola de videojuegos retro, utilizando una tarjeta controladora con un sistema operativo Linux base. El desarrollo prescinde por completo de entornos de escritorio o distribuciones preconfiguradas enfocadas en emulación, priorizando la configuración a bajo nivel de los componentes de software y hardware.

A través de este tutorial, se guiará al lector paso a paso para lograr un sistema en modo "Qiosco" que incluye la reproducción de una animación de arranque personalizada, la ejecución directa de una interfaz gráfica desarrollada en Python y la librería Pygame [1], operada exclusivamente mediante un gamepad, y la orquestación del emulador Mednafen [2] para ejecutar de forma nativa títulos de 8 y 16 bits (NES, SNES y Game Boy Advance). Adicionalmente, se implementará un mecanismo de automatización a nivel de núcleo (mediante reglas udev [3]) para la detección, sincronización y transferencia inteligente de archivos (ROMS) desde medios de almacenamiento USB extraíbles hacia la memoria interna.

2. Antecedentes teóricos

Para el desarrollo de la presente consola de videojuegos retro, es imperativo establecer los fundamentos teóricos que rigen el comportamiento del hardware y el software involucrado, con especial énfasis en el ecosistema de Linux embebido y los principios de emulación.

2.1. Sistemas embebidos y Linux embebido

Un sistema embebido se define como un sistema informático de propósito específico, diseñado para realizar una o varias funciones dedicadas, a menudo con restricciones de tiempo real, consumo de energía y tamaño [4]. A diferencia de las computadoras de propósito general, estos sistemas interactúan estrechamente con el hardware físico.

En este proyecto, se hace uso de una computadora de placa reducida (SBC) que ejecuta una distribución de Linux orientada a sistemas embebidos (Raspberry Pi OS Lite). El uso

de Linux embebido proporciona una abstracción del hardware mediante el núcleo (kernel), permitiendo la gestión eficiente de procesos, memoria y periféricos de entrada/salida (E/S) sin la sobrecarga de un entorno de escritorio gráfico [5].

2.2. Gestión de servicios y dispositivos en Linux

El funcionamiento autónomo de la consola depende de dos componentes críticos del espacio de usuario en sistemas Linux modernos: *systemd* y *udev*.

Por un lado, *systemd* es un administrador de sistemas y servicios que actúa como el proceso *init* (PID 1). Su función principal es inicializar los componentes del sistema tras el arranque del núcleo y mantener los servicios (daemons) en ejecución [6]. En el contexto de este proyecto, se utiliza para automatizar la ejecución de la interfaz gráfica desarrollada en Pygame sin intervención del usuario.

Por otro lado, el administrador de dispositivos *udev* es el responsable de gestionar los nodos de dispositivos en el directorio `/dev` dinámicamente. Permite al sistema operativo detectar la inserción o remoción de hardware en caliente (hot-plugging), como memorias USB o gamepads, y ejecutar scripts (reglas) específicos en respuesta a estos eventos [6]. Esto es el fundamento lógico que permite la transferencia automatizada de archivos ROM.

2.3. Emulación de Hardware

La emulación es el proceso mediante el cual un sistema informático (el anfitrión) reproduce fielmente el comportamiento y la arquitectura interna de otro sistema informático diferente (el huésped) [7]. A diferencia de la simulación, que solo busca replicar el comportamiento externo, la emulación decodifica y ejecuta las instrucciones del procesador original, traduciéndolas a la arquitectura del hardware actual (en este caso, ARM).

Para lograr tiempos de respuesta adecuados y mantener las tasas de fotogramas originales de consolas como la NES o SNES, los emuladores requieren una sincronización precisa con los temporizadores del sistema operativo anfitrión y la gestión directa de los búferes de audio y video. Herramientas como Mednafen logran esto mediante la capa de abstracción SDL, optimizando la ejecución de múltiples núcleos de emulación en plataformas de bajos recursos.

3. Lista de materiales

Para la implementación de este proyecto, se requiere el siguiente hardware. Se incluyen opciones equivalentes en caso de no contar con los componentes exactos:

- **1x Tarjeta microcontroladora:** Raspberry Pi 4 Model B. (*Equivalente:* Raspberry Pi 3B+ o Raspberry Pi 5, tomando en cuenta que los puertos de video pueden variar).
- **1x Memoria MicroSD:** Capacidad mínima de 16 GB, clase 10 para garantizar velocidades de lectura fluidas. (*Equivalente:* Unidad de estado sólido SSD por USB).
- **1x Fuente de alimentación:** Adaptador de corriente USB-C de 5V y al menos 3A.

- **1x Mando de videojuegos (Gamepad):** Control DualShock 4 de PlayStation 4. (*Equivalente:* Cualquier control USB genérico o control de Xbox).
- **1x Cable USB tipo A a Micro-USB:** Para la conexión alambrada del control.
- **1x Memoria USB (Pendrive):** Utilizada para la transferencia de los juegos (ROMS).
- **1x Monitor o pantalla:** Con entrada VGA o HDMI.
- **1x Cable de video y adaptador:** Cable Micro-HDMI a HDMI, junto con un convertidor activo de HDMI a VGA (si el monitor es antiguo).
- **1x Sistema de audio:** Bocina externa genérica y un cable de audio Jack 3.5 mm (macho a macho).
- **1x Teclado USB genérico:** Necesario únicamente para la configuración inicial del sistema operativo por línea de comandos.

4. Descripción del funcionamiento de los componentes electrónicos

Para comprender cómo interactúa el sistema, es importante conocer la función de los periféricos conectados a la tarjeta central:

4.1. Almacenamiento (MicroSD y memoria USB)

Estos dispositivos utilizan memoria tipo Flash NAND [8], un tipo de almacenamiento no volátil que retiene la información incluso sin energía. En este proyecto, la tarjeta MicroSD actúa como el "disco duro" de la consola, albergando el sistema operativo Linux y los emuladores. La memoria USB sirve como un puente externo; el sistema detecta sus variaciones de voltaje al insertarla y activa un protocolo automático para extraer los archivos de juegos (ROMS).

4.2. Conversor de video (HDMI a VGA)

Este es uno de los componentes más importantes si se utilizan monitores antiguos. La Raspberry Pi emite señales de video en formato HDMI, el cual es puramente digital (secuencias de ceros y unos). Sin embargo, el estándar VGA requiere señales analógicas (ondas continuas de voltaje). El convertidor contiene un pequeño circuito integrado en su interior que decodifica la señal digital y la transforma en analógica en tiempo real [8] para que el monitor pueda dibujar la imagen.

4.3. Interfaz de entrada (DualShock 4)

El gamepad actúa como la interfaz humano-máquina principal. Internamente, cuenta con interruptores táctiles bajo los botones y potenciómetros analógicos en las palancas. Al presionar un botón, se cierra un circuito eléctrico. Un microcontrolador interno en el mando lee este cierre y envía un paquete de datos digitales a través del cable USB hacia la Raspberry Pi, indicando exactamente qué botón fue accionado.

4.4. Interfaz de salida de audio (Jack 3.5mm y bocina)

A diferencia del video que requiere un conversor externo para VGA, la Raspberry Pi ya cuenta con un conversor digital-analógico (DAC) [9] para el audio. Este componente envía pulsos eléctricos a través del cable Jack 3.5 mm hacia la bocina externa. El electroimán dentro de la bocina vibra en respuesta a estos voltajes, empujando el aire y creando las ondas sonoras que el usuario escucha.

5. Descripción de la tarjeta controladora

El cerebro de esta consola retro es una **Raspberry Pi 4 Model B**. A diferencia de los microcontroladores básicos (como un Arduino o un PIC), esta placa es una computadora completa de placa reducida (SBC, por sus siglas en inglés) [9]. Sus componentes principales son:

- **SoC (System on a Chip):** Utiliza un chip Broadcom BCM2711. En una sola pastilla de silicio integra el procesador principal (un ARM Cortex-A72 de cuatro núcleos) y el procesador gráfico (VideoCore VI) [9]. Este último es crucial para el proyecto, ya que se encarga de renderizar los gráficos del menú y emular las consolas de 8 y 16 bits.
- **Memoria RAM (LPDDR4):** Es la memoria de trabajo a corto plazo. Aquí se cargan las instrucciones del sistema operativo Linux y se descomprimen los juegos antes de ser enviados al procesador.
- **Controlador USB y de red:** Un bus dedicado que permite la conexión de los periféricos (teclado, control y memoria USB) sin generar cuellos de botella en la transferencia de datos.
- **Puertos de Entrada/Salida:** La placa expone puertos físicos esenciales para la consola, destacando los puertos USB (2.0 y 3.0), dos salidas Micro-HDMI (para video de alta definición), el conector Jack 3.5 mm para audio analógico, y el puerto de alimentación USB-C.

Aunque la tarjeta cuenta con un cabezal de pines GPIO (General Purpose Input/Output) diseñado para controlar electrónica a bajo nivel (como LEDs o motores), para los fines de este emulador, la interacción se ha confinado a los puertos periféricos estándar, maximizando la seguridad y facilidad de ensamblaje para el usuario novato.



Figura 1: Raspberry Pi 4 Model B

6. Seguridad y cuidado del equipo

6.1. Cuidado de la salud y advertencias de riesgos

Aunque los sistemas embebidos operan con voltajes muy bajos (5V en corriente continua), lo cual no representa un riesgo de electrocución fatal para el ser humano, es imperativo observar precauciones básicas:

- **Fuente de alimentación:** El adaptador de corriente se conecta a la red eléctrica (120V CA). Nunca manipule el adaptador con las manos mojadas ni utilice cables pelados o dañados.
- **Riesgo de quemaduras (Térmico):** El procesador (SoC) de la Raspberry Pi realiza millones de cálculos por segundo durante la emulación, lo que genera calor. Bajo estrés continuo, este componente puede superar los 70°C. Evite tocar directamente los chips de la placa mientras esté energizada o inmediatamente después de apagarla para evitar quemaduras leves.

6.2. Cuidado de los componentes electrónicos delicados

El hardware de este proyecto es susceptible a daños invisibles. Para garantizar la vida útil de la consola, se deben seguir estas reglas:

- **Descargas Electroestáticas (ESD):** El cuerpo humano acumula electricidad estática. Una descarga imperceptible para nosotros puede destruir los circuitos lógicos de la Raspberry Pi. Antes de manipular la tarjeta, toque una superficie metálica conectada a tierra (como el marco de una ventana) o utilice una pulsera anti-estática. Sostenga la placa siempre por los bordes.

- **Integridad de la Memoria (MicroSD):** La tarjeta MicroSD es el componente más frágil a nivel de software. Si se corta la energía de la placa repentinamente mientras el sistema operativo está escribiendo datos, la memoria se corromperá. Por ello, la consola cuenta con una función de apagado seguro integrada en el control; **nunca** desconecte el cable de corriente sin antes utilizar dicha función.
- **Gestión de Temperatura:** Para evitar que el procesador reduzca su rendimiento para protegerse del calor (*Thermal Throttling*), se recomienda encarecidamente instalar disipadores térmicos pasivos de aluminio sobre el SoC y el chip de memoria RAM.

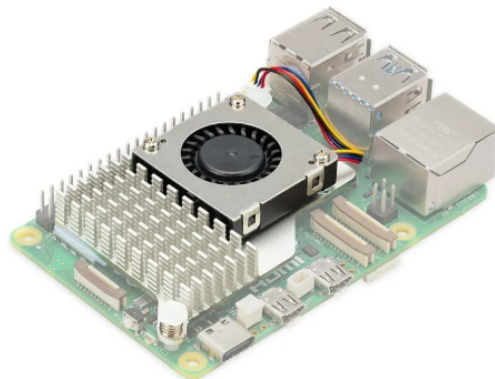


Figura 2: Ubicación correcta de los disipadores térmicos en la Raspberry Pi 4.

7. Configuración de la tarjeta controladora

Esta sección detalla el proceso de preparación del software base, asumiendo que la tarjeta MicroSD ha sido formateada previamente. El objetivo es configurar un entorno Linux puramente de consola (sin escritorio) y prepararlo para interactuar con el hardware de video y los periféricos.

7.1. Instalación del Sistema Operativo base

Para cumplir con el requerimiento de no utilizar gestores de ventanas ni distribuciones preconfiguradas, se optó por instalar **Raspberry Pi OS Lite** (basado en Debian) [10]. Esta versión carece de entorno gráfico (X11 o Wayland), lo que libera una cantidad considerable de memoria RAM y ciclos de procesamiento que serán aprovechados íntegramente por el emulador.

El flasheo de la imagen en la memoria MicroSD se realiza mediante la herramienta oficial *Raspberry Pi Imager* [10], asegurándose de activar el acceso por SSH (Secure Shell) [11] en los ajustes avanzados para permitir la configuración remota desde otra computadora.

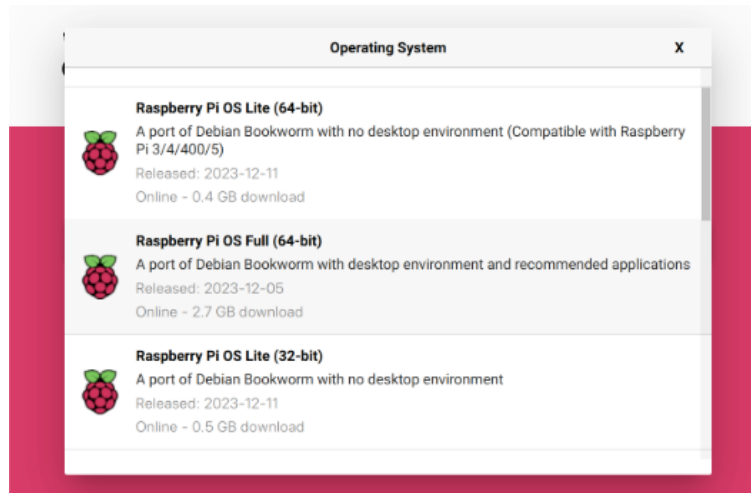


Figura 3: Selección del sistema operativo base sin entorno de escritorio.

7.2. Actualización e instalación de dependencias

Una vez que el sistema arranca y se establece la conexión SSH, el primer paso es actualizar la lista de repositorios del gestor de paquetes `apt` [11] para asegurar la obtención de las versiones más recientes. Posteriormente, se instalan los tres pilares del software de nuestra consola:

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3 python3-pygame mednafen -y
```

- **python3 y python3-pygame:** Permiten ejecutar y renderizar la interfaz gráfica (menú de selección) directamente sobre el hardware [1].
- **mednafen:** Es el núcleo emulador multisistema por línea de comandos, elegido por su alta precisión y bajo consumo de recursos para consolas de 8 y 16 bits [2].

7.3. Configuración del subsistema de video (KMS/DRM)

Al no contar con un entorno de escritorio que dibuje ventanas, los programas necesitan una forma de "hablar" directamente con la tarjeta de video. Para ello, se habilita el controlador KMS (Kernel Mode Setting) y la API DRM (Direct Rendering Manager).

Esto se configura accediendo a la herramienta de configuración de la placa mediante el comando `sudo raspi-config` [10], navegando a *Advanced Options - GL Driver* y seleccionando *GL (Fake KMS)* o *Full KMS* según la compatibilidad del kernel.

Gracias a esto, Python y Mednafen pueden usar la pantalla de forma nativa. Para que la librería gráfica sepa que debe usar este controlador, se define la siguiente variable de entorno en el sistema [12]:

```
export SDL_VIDEODRIVER=kmsdrm
```

7.4. Automatización y gestión de medios extraíbles (udev)

Para cumplir con el requisito de expandir la biblioteca de juegos de forma dinámica al insertar una memoria USB, se configuró el gestor de dispositivos dinámicos de Linux, conocido como `udev` [3].

Para lograr una experiencia interactiva que no interrumpa abruptamente al usuario, se diseñó una arquitectura de paso de mensajes. Se creó una regla personalizada en la ruta `/etc/udev/rules.d/99-usb-sync.rules`. Esta regla le indica al núcleo de Linux que, en el instante en que detecte la conexión de un dispositivo de almacenamiento masivo (*block*), ejecute una instrucción ligera para crear un archivo temporal que servirá como semáforo o bandera (*flag*):

```
ACTION=="add", SUBSYSTEM=="block", ENV{ID_FS_USAGE}=="filesystem",  
RUN+="/bin/touch /tmp/usb_pending"
```

La existencia del archivo `/tmp/usb_pending` es detectada por el hilo principal de la interfaz gráfica (Pygame), el cual detiene su renderizado normal para desplegar una ventana emergente solicitando la confirmación del usuario.

Únicamente si el usuario aprueba la transferencia mediante el *gamepad*, la interfaz invoca en segundo plano al script `sync_roms.sh`. Este script se encarga de montar la memoria USB temporalmente y buscar archivos con extensiones específicas (`.nes`, `.sfc`, `.gba`). Utiliza el comando `rsync` [11] para transferir los juegos a la tarjeta MicroSD de forma inteligente, copiando únicamente los archivos nuevos y evitando sobrescribir duplicados.

Finalmente, el script documenta los archivos transferidos en `/tmp/juegos_copiados.txt` para proveer retroalimentación visual en la interfaz, y procede a desmontar la unidad de forma segura.

8. Desarrollo de los componentes de software

Para dotar de vida a la consola, se desarrolló una arquitectura de software dividida en tres componentes principales: el frontend (interfaz de usuario), el backend (motor de emulación) y el orquestador (script maestro).

8.1. Interfaz gráfica (frontend en Python)

Se programó un menú interactivo utilizando el lenguaje Python y la librería `Pygame` [1]. Este script (`menu.py`) tiene la responsabilidad exclusiva de interactuar con el usuario.

El programa inicia escaneando el directorio local en busca de archivos ROMS válidos y los despliega en una lista con desplazamiento dinámico (*scroll*). A través del módulo `pygame.joystick` [1], el script escucha los eventos físicos del control de PlayStation 4. Cuando el usuario presiona el botón de confirmación, el programa realiza una acción crítica: no lanza el juego directamente, sino que anota la ruta del juego seleccionado en un archivo de texto temporal (`/tmp/next_game.txt`) y procede a cerrarse completamente (`sys.exit()`).

Esta decisión de diseño es vital para liberar el uso de la memoria RAM y, lo más importante, soltar el controlador de video KMS/DRM [12], el cual no puede ser utilizado por dos aplicaciones al mismo tiempo.

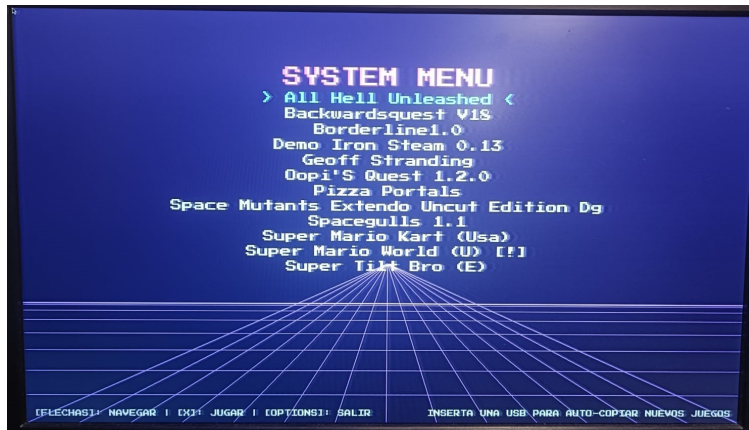


Figura 4: Interfaz gráfica principal desarrollada en Python.

```

1
2 # [Configuracion y variables de entorno omitidas por brevedad]
3 ROM_DIR = "/home/pi/roms"
4 AUDIO_USB = "/home/pi/assets/moneda.wav"
5
6 def main():
7     pygame.init()
8     pygame.mixer.init()
9     screen = pygame.display.set_mode((0, 0), pygame.FULLSCREEN)
10
11 # [Carga de fuentes, sonidos y joystick omitida]
12
13 corriendo = True
14 while corriendo:
15
16     # --- INTERRUPCION: VERIFICACION DE USB ---
17     # El sistema verifica si udev levanto la bandera temporal
18     if os.path.exists('/tmp/usb_pending'):
19         # [Llamadas a Pygame para dibujar la ventana omitidas]
20
21         if sonido_alerta:
22             sonido_alerta.play()
23
24             esperando_respuesta = True
25             while esperando_respuesta:
26                 for event in pygame.event.get():
27                     if event.type == pygame.JOYBUTTONDOWN:
28                         if event.button == 0: # Boton X (SI)
29                             # Ejecutar script de copia real en bash
30                             subprocess.run(["sudo", "/usr/local/bin/sync_roms.sh"])
31
32         # [Logica de lectura de /tmp/juegos_copiados.txt omitida]
33         esperando_respuesta = False
34
35         # Limpiar bandera y reiniciar el ciclo
36         os.remove('/tmp/usb_pending')
37         juegos = obtener_juegos()

```

```

38         continue
39
40     # --- LOGICA NORMAL DEL MENU ---
41     # [Dibujo de fondo Synthwave y renderizado de texto omitido]
42
43     # --- MANEJO DE EVENTOS (SELECCION DE JUEGO) ---
44     for evento in pygame.event.get():
45         if evento.type == pygame.JOYBUTTONDOWN:
46             if evento.button == 0 and juegos:
47                 # Escribir la seleccion para el orquestador
48                 ruta_juego = os.path.join(ROM_DIR, juegos[indice_seleccionado])
49                 with open("/tmp/next_game.txt", "w") as f:
50                     f.write(ruta_juego)
51
52                 # Salir para que Mednafen tome el control
53                 pygame.quit()
54                 sys.exit(0)
55
56             elif evento.button == 9: # Boton OPTIONS
57                 subprocess.run(["sudo", "poweroff"])
58                 sys.exit(0)
59
60 if __name__ == "__main__":
61     main()

```

Código 1: Lógica principal de la interfaz y detección de eventos USB en Pygame.

Adicionalmente, se integró el módulo `pygame.mixer` para proporcionar retroalimentación auditiva. Al tratarse de un sistema embebido que carece de periféricos de entrada tradicionales (teclado/ratón), el audio sirve como un canal crucial para informar al usuario sobre cambios de estado en el sistema (por ejemplo, la inserción de un dispositivo externo).

8.2. Motor de emulación (Mednafen)

El núcleo encargado de ejecutar los juegos es **Mednafen** [2]. Se seleccionó por ser un emulador impulsado íntegramente por línea de comandos, lo que lo hace perfecto para sistemas sin escritorio. Mednafen recibe como argumento la ruta exacta del juego; toma el control del hardware de video, inicializa los controladores de audio y mapea las entradas del gamepad para ejecutar el título a 60 cuadros por segundo constantes.

8.3. El orquestador (script maestro en Bash)

Para unir el frontend y el motor de emulación sin conflictos, se desarrolló un script maestro en Bash (`start.sh`) [11]. Este script actúa como el verdadero cerebro de la consola mediante un ciclo infinito (`while true`).

Su flujo de trabajo es el siguiente:

1. Limpia la pantalla de la terminal (`clear`) para ocultar el texto de Linux.
2. Ejecuta el menú de Python y pausa sus propias operaciones hasta que el usuario elija un juego y Python se cierre.

3. Lee el archivo temporal `/tmp/next_game.txt`.
4. Ejecuta **Mednafen** pasando el juego elegido. Además, silencia cualquier salida de texto de diagnóstico enviándola a un agujero negro de datos (`>/dev/null 2>&1`) [11] para no romper la inmersión visual.
5. Una vez que el usuario sale del juego (presionando los atajos de teclado o botones correspondientes), Mednafen se cierra y el ciclo del script maestro se reinicia, volviendo al paso 1 para mostrar el menú nuevamente.

```
1  #!/bin/bash
2  export SDL_VIDEODRIVER=kmsdrm
3  export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
4
5
6  while true; do
7  rm -f /tmp/next_game.txt
8
9  # Limpiamos por completo la pantalla y el texto viejo
10 clear
11 sleep 1
12
13 python3 /home/pi/menu.py
14
15 if [ -f /tmp/next_game.txt ]; then
16 JUEGO=$(cat /tmp/next_game.txt)
17
18 # Volvemos a limpiar antes de arrancar el juego
19 clear
20
21 # El "> /dev/null 2>&1" absorbe todo el texto de Mednafen para que no se
    vea
22 mednafen "$JUEGO" > /dev/null 2>&1
23
24 else
25 clear
26 break
27 fi
28 done
```

Código 2: Script maestro en bash (start.sh)

8.4. Manejo de eventos externos: interrupción por USB

El flujo principal del menú desarrollado en Python (*main loop*) fue diseñado para monitorear constantemente el sistema de archivos en busca de la bandera o semáforo `/tmp/usb_pending`, generada por el subsistema *udev*.

En la versión final, cuando se detecta esta bandera, el sistema detiene la renderización del menú de juegos y ejecuta una rutina de interrupción visual y sonora. Primero, se reproduce un efecto de audio de alerta y posteriormente se dibuja una ventana emergente interactiva.

Para asegurar que la interfaz sea robusta ante nombres de archivos atípicamente largos o una gran cantidad de archivos transferidos, se implementaron las siguientes medidas de control en la ventana emergente:

- **Redimensionamiento de ventana:** Se establecieron dimensiones fijas de 1100×650 píxeles, garantizando espacio suficiente para el texto sin desbordar la resolución nativa de la pantalla.
- **Centrado dinámico:** Mediante el método `get_width()` de Pygame, las coordenadas de renderizado de los textos se calculan en tiempo de ejecución, asegurando un centrado perfecto sin importar la longitud de la cadena.
- **Truncamiento de cadenas:** Se limitó la previsualización de juegos copiados a un máximo de 10 elementos. Además, se introdujo una validación que trunca cualquier nombre de archivo que exceda los 55 caracteres, concatenando puntos suspensivos para evitar el desbordamiento visual fuera de los límites de la ventana emergente.

Una vez que el usuario confirma la acción mediante el gamepad, el menú invoca al script Bash subyacente mediante el módulo `subprocess`, delegando la carga computacional de la transferencia al sistema operativo y esperando la señal de finalización para actualizar la lista de *ROMs* disponibles en pantalla.

9. Integración en una solución de hardware y software

La integración final del sistema consiste en lograr que, al energizar la placa, esta no pida un inicio de sesión por texto ni muestre un entorno de escritorio, sino que actúe como un electrodoméstico dedicado (lo que en ingeniería se conoce como *Kiosk Mode* o Modo Quiosco). Para lograr esto, se modificó el comportamiento de arranque a nivel del núcleo del sistema operativo.

9.1. El gestor de servicios (systemd)

En las distribuciones modernas de Linux, el programa que controla qué aplicaciones arrancan junto con el sistema operativo se llama **systemd** [13]. En lugar de ejecutar el código de la consola manualmente, se delegó esta responsabilidad creando un servicio del sistema personalizado.

Se generó un archivo de configuración en la ruta `/etc/systemd/system/retromenu.service`. Este archivo contiene directivas estrictas que le indican al sistema operativo cómo debe orquestar nuestra consola [13]:

- **ExecStart:** Define el comando principal a ejecutar una vez que el sistema enciende. En nuestro diseño, esta directiva apunta directamente al script orquestador (`/bin/bash /home/pi/start.sh`).
- **User:** Se especificó que el servicio arranque bajo el usuario `pi` (usuario estándar) y no como superusuario (`root`). Esta es una buena práctica de ciberseguridad, ya que confina los permisos del emulador y la lectura de la memoria USB a un entorno seguro.

- **Restart=always:** Funciona como un mecanismo de tolerancia a fallos. Si el script maestro o la interfaz de Python llegaran a colapsar por una lectura corrupta de memoria, **systemd** detectará la caída y volverá a arrancar la interfaz gráfica instantáneamente de forma automática.
- **Control de pantalla (TTY):** Se configuró el servicio para que tome el control absoluto de la primera terminal de video (**tty1**). Esto bloquea la entrada de texto estándar de Linux y cede la administración de los píxeles al controlador KMS y a la librería Pygame.

```
[Unit]
Description=Menu de Consola Retro en Python
# Espera a que el sistema base y tu video de arranque terminen
After=multi-user.target splash.service
# Evita que la terminal de texto normal pelee por el control de la pantalla o el
# teclado
Conflicts=getty@tty1.service

[Service]
Type=simple
User=pi
Group=pi
# Le damos control de la terminal 1 para que Pygame pueda dibujar y leer eventos
# libremente
StandardInput=tty
StandardOutput=tty
TTYPath=/dev/tty1
# Ejecuta tu script
ExecStart=/bin/bash /home/pi/start.sh
# Si el script falla o se cierra por accidente, se vuelve a abrir solo
Restart=always
RestartSec=2

[Install]
WantedBy=multi-user.target
```

Figura 5: Configuración del servicio **systemd** para el modo Quiosco y tolerancia a fallos.

9.2. Secuencia de auto-arranque y animación

Para completar la experiencia de usuario y ocultar el texto de diagnóstico (*bootlog*) que Linux imprime al encender, la secuencia de arranque se ajustó para reproducir primero un video o logotipo personalizado justo antes de cederle el control de la pantalla a nuestro software.

La integración funciona como una carrera de relevos: el sistema enciende, el hardware de video proyecta la animación de arranque, y en el milisegundo en que esta finaliza, **systemd** toma el relevo e inicia nuestro servicio **retromenu.service**. En este punto, el orquestador **start.sh** entra en su ciclo infinito, limpia cualquier residuo en la pantalla e invoca la interfaz gráfica de selección de juegos.

Gracias a esta orquestación de bajo nivel, el usuario novato únicamente necesita conectar la consola a la corriente eléctrica y tomar el gamepad, sin requerir jamás el uso de un teclado o la interacción manual con la terminal de comandos.

10. Solución a problemas comunes

Durante el proceso de diseño, desarrollo e integración de una consola de videojuegos en un sistema embebido, es habitual enfrentarse a fallos lógicos derivados de la concurrencia de hardware, el aislamiento de permisos de seguridad y la gestión de variables de entorno en sistemas operativos sin entorno gráfico. A continuación, se detallan los problemas más recurrentes y sus soluciones técnicas paso a paso.

10.1. Error: `pygame.error: kmsdrm not available` (condición de carrera)

Síntoma: Al intentar regresar al menú principal de selección inmediatamente después de haber cerrado un juego, la pantalla permanece en negro o la terminal despliega de forma cíclica un fallo crítico indicando que el driver gráfico KMS/DRM no se encuentra disponible.

Causa: Este comportamiento responde a una condición de carrera por el control de la memoria de video. El subsistema de renderizado directo KMS/DRM de Linux es exclusivo: solo permite una aplicación activa en pantalla a la vez. Si la interfaz gráfica en Python intenta inicializarse en la memoria RAM un milisegundo antes de que el proceso del emulador Mednafen haya terminado de liberar los búferes de la GPU, la librería Pygame chocará contra un bloqueo de hardware y colapsará instantáneamente.

Solución:

1. Garantizar un cierre absoluto y limpio en el código de Python invocando explícitamente los comandos `pygame.quit()` y `sys.exit(0)` al momento de seleccionar un título.
2. Implementar una barrera de tiempo o pausa táctica en el script orquestador maestro en Bash (`start.sh`). La inserción del comando `sleep 1` justo antes de reiniciar el bucle del menú otorga al núcleo de Linux el margen necesario para limpiar la tarjeta de video y reconfigurar la resolución nativa.

10.2. Error: `mednafen: command not found` en ejecución por `systemd`

Síntoma: El sistema operativo ejecuta y opera la interfaz del menú de manera fluida en pruebas manuales por SSH. Sin embargo, al arrancar la consola de forma automática en el encendido, elegir un videojuego provoca un parpadeo visual y el sistema regresa de inmediato a la interfaz sin cargar la emulación. Los registros del sistema exponen la línea: `start.sh: line 18: mednafen: command not found`.

Causa: Al establecer una sesión remota por SSH, Linux hereda un entorno de usuario completo que incluye la variable `$PATH`, mapeando carpetas especiales como `/usr/games/`, que es donde reside el ejecutable de Mednafen de forma nativa. Por el contrario, los servicios administrados en segundo plano por el demonio `systemd` operan en un entorno de aislamiento restringido por seguridad, el cual carece de la ruta de directorios de juegos en su mapa de búsqueda por defecto.

Solución: Es necesario instruir explícitamente al script orquestador sobre la ubicación de los binarios del sistema. Esto se soluciona declarando la variable de entorno global en la parte superior del archivo `start.sh`:

```
export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games
```

10.3. Fallo de interpretación: `SyntaxError: invalid syntax` en scripts de Bash

Síntoma: Al procesar modificaciones en el flujo del sistema o intentar inicializar manualmente el archivo orquestador, la consola detiene la ejecución reportando un error de sintaxis en líneas que involucran comandos nativos de Linux como `export SDL_VIDEODRIVER=kmsdrm`.

Causa: Este error se suscita cuando el sistema operativo intenta interpretar un script escrito bajo la estructura y sintaxis de Bash a través del compilador o motor de ejecución de Python (por ejemplo, al digitar erróneamente en consola `python3 start.sh`).

Solución:

1. Para verificar el comportamiento manualmente desde la terminal, se debe invocar el intérprete correcto usando el comando: `bash /home/pi/start.sh`.
2. Para el despliegue automático del Modo Quiosco, es crucial examinar el archivo del servicio con `sudo nano /etc/systemd/system/retromenu.service` y constatar que la directiva de ejecución no contenga referencias a Python, quedando estructurada de la siguiente manera:

```
ExecStart=/bin/bash /home/pi/start.sh
```

10.4. Bloqueo e inactividad al invocar la función de apagado seguro

Síntoma: Al presionar el botón físico asignado en el mando de control para apagar el dispositivo, la interfaz gráfica limpia los elementos visuales y despliega exitosamente el aviso “APAGANDO SISTEMA...”, pero la tarjeta de desarrollo permanece encendida de forma indefinida y no corta el suministro de energía.

Causa: Por motivos de seguridad e integridad del sistema de archivos, el núcleo de Linux restringe las instrucciones de detención de hardware (`poweroff`) exclusivamente al superusuario. Al mandar la instrucción desde Python a través de `subprocess.run(["sudo", "poweroff"])`, el sistema operativo pausa la ejecución en segundo plano, quedando a la espera de que el usuario digite la contraseña de administrador. Al estar la consola configurada en modo Quiosco y sin una terminal de comandos interactiva en pantalla, el proceso entra en un bucle de espera infinito.

Solución: Configurar una regla de excepción controlada y específica para la instrucción de apagado seguro en el archivo maestro de privilegios. Esto se efectúa abriendo el editor seguro del sistema mediante el comando:

```
sudo EDITOR=nano visudo
```

Posteriormente, se navega hasta el final absoluto del documento y se anexa la siguiente directiva, la cual exime al usuario operativo de ingresar credenciales únicamente para dicha tarea:

```
pi ALL=(ALL) NOPASSWD: /sbin/poweroff, /usr/sbin/poweroff
```

11. Conclusiones

El desarrollo de este proyecto demostró que la implementación de un sistema embebido trasciende la simple conexión de componentes electrónicos; requiere una orquestación minuciosa entre el hardware y las capas más profundas del sistema operativo. Al prescindir deliberadamente de entornos de escritorio convencionales y distribuciones pre-empaquetadas, se logró una comprensión exhaustiva de la arquitectura de Linux.

Desde la perspectiva de la administración del sistema operativo, el mayor reto y aprendizaje radicó en el manejo de la concurrencia y la automatización. La configuración del controlador KMS/DRM evidenció cómo los recursos de video a bajo nivel son altamente restrictivos y requieren un control estricto de los procesos activos para evitar condiciones de carrera. Asimismo, la implementación de reglas `udev` y la creación de demonios personalizados mediante `systemd` comprobaron que es posible dotar a una computadora de propósito general de la robustez, seguridad y transparencia operativa de un electrodoméstico dedicado (Modo Quiosco).

En conclusión, se logró cumplir con todos los requerimientos de la rúbrica, obteniendo una consola estable, eficiente en el uso de memoria RAM y completamente autónoma, demostrando que el dominio de los comandos de Bash y la estructura interna de Linux es fundamental para el desarrollo profesional de dispositivos embebidos.

12. Cuestionario

A modo de repaso para el lector que busque replicar este tutorial, se plantean las siguientes preguntas técnicas:

1. ¿Por qué es perjudicial utilizar un entorno de escritorio (X11/Wayland) en un proyecto de emulación pura?

Porque los entornos de escritorio consumen continuamente ciclos de reloj del procesador y memoria RAM para mantener la interfaz gráfica (ventanas, cursores, barras de tareas) en segundo plano, restando recursos vitales que el emulador necesita para mantener los 60 cuadros por segundo constantes, además de arruinar la inmersión visual del usuario.

2. ¿Qué función cumple la regla `udev` en el sistema?

Es un subsistema del núcleo de Linux que escucha eventos físicos de hardware. En nuestro proyecto, actúa como un "gatillo" que detecta en tiempo real la inserción de una memoria USB y dispara automáticamente el script de sincronización de juegos sin necesidad de que el usuario intervenga por consola.

3. ¿Cuál es la diferencia entre arrancar un script manualmente por SSH y hacerlo mediante `systemd`?

Al arrancar manualmente, el script hereda los permisos y el entorno visual del usuario remoto, y morirá si se cierra la conexión. Al usar `systemd`, el script se convierte en un

servicio nativo del sistema que arranca automáticamente al energizar la placa, se aísla por seguridad y cuenta con la capacidad de reiniciarse solo en caso de una falla crítica.

Bibliografía de consulta

- [1] *Pygame Front Page - Documentation*, Pygame Community, 2024. dirección: <https://www.pygame.org/docs/>
- [2] M. Team, *Mednafen Emulator Documentation*, 2024. dirección: <https://mednafen.github.io/documentation/>
- [3] *udev(7) — Linux manual page*, Freedesktop.org, 2023. dirección: <https://www.freedesktop.org/software/systemd/man/udev.html>
- [4] S. Heath, *Embedded Systems Design*, 2.^a ed. Newnes, 2003.
- [5] K. Yaghmour, J. Masters, G. Ben-Yossef y P. Gerum, *Building Embedded Linux Systems*, 2.^a ed. O'Reilly Media, 2008.
- [6] R. Love, *Linux Kernel Development*, 3.^a ed. Addison-Wesley Professional, 2010.
- [7] N. Altice, *I Am Error: The Nintendo Family Computer / Entertainment System Platform*. MIT Press, 2015.
- [8] T. L. Floyd, *Fundamentos de Sistemas Digitales*, 11th. Pearson Educación, 2016.
- [9] *Raspberry Pi 4 Tech Specs*, Raspberry Pi Ltd, 2019. dirección: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications/>
- [10] *Raspberry Pi OS Documentation*, Raspberry Pi Ltd, 2024. dirección: <https://www.raspberrypi.com/documentation/computers/os.html>
- [11] W. E. Shotts, *The Linux Command Line: A Complete Introduction*, 2nd. No Starch Press, 2019.
- [12] *SDL2 Documentation: Video Data*, Simple DirectMedia Layer, 2024. dirección: <https://wiki.libsdl.org/SDL2/FrontPage>
- [13] *systemd.service — Service unit configuration*, Freedesktop.org, 2023. dirección: <https://www.freedesktop.org/software/systemd/man/systemd.service.html>